

Data Extraction Reliability & Data Integrity

E/21/327

Induka Rathnayake

Data Extraction Reliability

When building an automated fault classification system for power distribution networks, the deep learning model is only as smart as the data it trains on. If the extracted time-series data like high-frequency voltage and current transients is missing points, delayed or out of order, the neural network will learn incorrect electrical signatures.



[Source - <https://electrical-engineering-portal.com/eight-most-common-distribution-network-faults/>]

Let's outline the metrics and methods required to mathematically prove that our data extraction pipeline is 100% reliable.

1. Defining Extraction Reliability in Chronological Systems :-)

In standard databases, reliability just means the data arrives safely. In time-series systems, sequence is everything.

Time-series data is defined by its strict chronological order. Extraction reliability therefore rests on three pillars,

- Completeness - Did every single sampled measurement arrive?
- Temporal Integrity - Did it arrive in the exact order it occurred?
- Latency/Throughput - Did it arrive fast enough to prevent memory crashes on the physical device?

2. Data Completeness Metrics :-)

In deep learning, missing data leads to silent corruption. An algorithm might still run, but the output will be completely wrong.

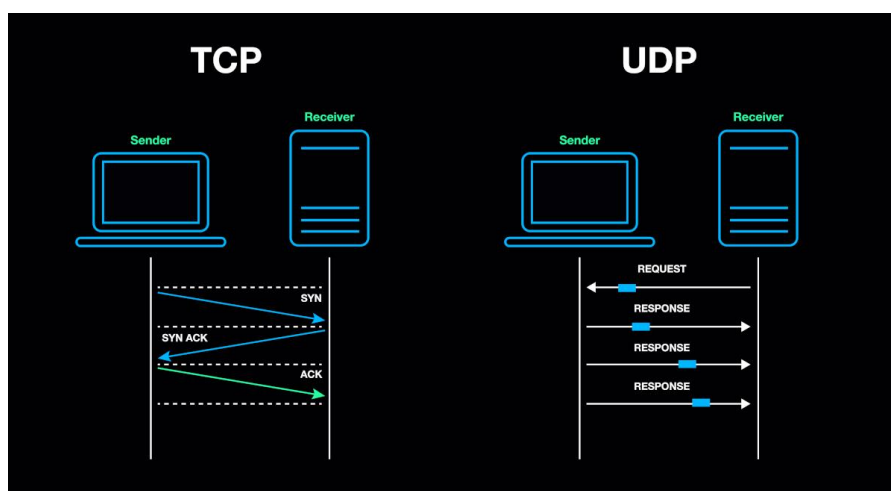
Packet Loss Rate (PLR)

When streaming data from a relay or simulation, data is sent in packets. PLR is the percentage of packets that fail to reach the destination.

- Zero-Loss Guarantees - For analyzing fault transients (which happen in milliseconds). So we need a 0% PLR during the fault window.

If we drop packets during a voltage sag, deep learning model **might misclassify** the fault type entirely.

- Mitigation - To achieve zero-loss, pipelines use robust buffering methods (like Apache Kafka topics) or reliable network protocols (TCP over UDP) to guarantee dropped packets are re-transmitted.



[Source - <https://obkio.com/blog/what-is-packet-reordering/>]

3. Temporal & Chronological Integrity :-)

When monitoring a distribution network, data often comes from multiple sensors or simulated nodes at once. Keeping their timing perfectly synchronized is a massive challenge.

Clock Drift and Jitter

Hardware clocks are imperfect. Two separate microcontrollers might drift apart by a few microseconds over an hour. When combining their data to calculate phase angles, this drift looks like an actual electrical anomaly.

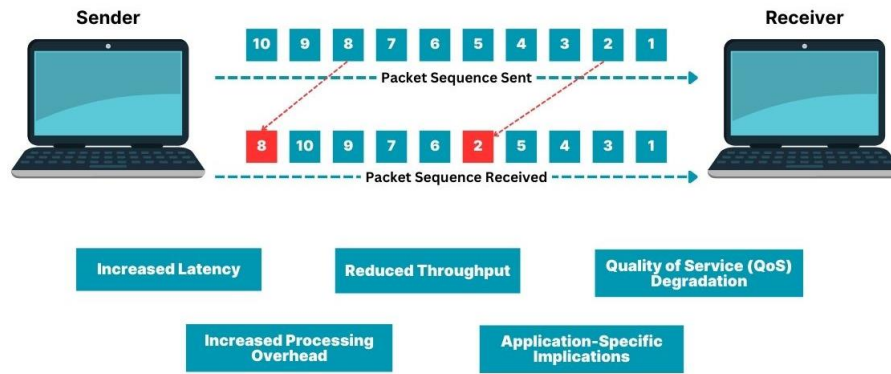
- Solution - Use Precision Time Protocol (PTP - IEEE 1588) or GPS-synchronized timestamps at the exact moment of sampling before the data is exported.

Out-of-Order Packets & Watermarking

In a network, Packet #5 might take a faster route than Packet #4, arriving out of sequence.

- Watermarking - Streaming systems inject watermarks (timestamp checkpoints) into the stream.

The receiving database will wait in a buffer until all packets before that watermark have arrived before feeding the data into the deep learning model.



[Source - <https://obkio.com/blog/what-is-packet-reordering/>]

4. Throughput, Latency and Pipeline Backpressure :-)

During a steady-state operation, data flow is calm. During a fault, intelligent electronic devices (IEDs) suddenly trigger high-frequency recording, causing a massive spike in data volume.

4.1 Handling Backpressure

If the data destination (database or AI inference engine) cannot process incoming data fast enough, it pushes back against the network. This is called backpressure.

If a device lacks the RAM to handle this backpressure, its internal memory buffers will overflow and needed fault data will be permanently overwritten and lost.

Real-world export methods must include flow-control logic to pause, compress or prioritize critical fault flags over standard telemetry.

5. Statistical Validation of Extraction Sameness :-)

How to prove in the FYP that the data exported is an **exact replica of the original simulation** or physical measurement? Statistical math should be used.

5.1 Root-Mean-Square Error (RMSE)

The error between the source matrix (in PSCAD/MATLAB) and the final exported file (CSV/Parquet) is calculated.

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (Predicted_i - Actual_i)^2}{N}}$$

[Source - <https://towardsdatascience.com/regression-an-explanation-of-regression-metrics-and-what-can-go-wrong-a39a9793d914/>]

If lossless extraction is used, the RMSE must be exactly 0.00. If a lossy compression method is used to save space, RMSE helps prove that the distortion is negligible.

5.2 Cross-Correlation Analysis

This measures how well the exported waveform aligns with the original waveform in the time domain.

Pearson's correlation coefficient when applied to a **sample** is commonly represented by r_{xy} and may be referred to as the **sample correlation coefficient** or the **sample Pearson correlation coefficient**. We can obtain a formula for r_{xy} by substituting estimates of the covariances and variances based on a **sample** into the formula above. Given paired data $\{(x_1, y_1), \dots, (x_n, y_n)\}$ consisting of n pairs, r_{xy} is defined as:

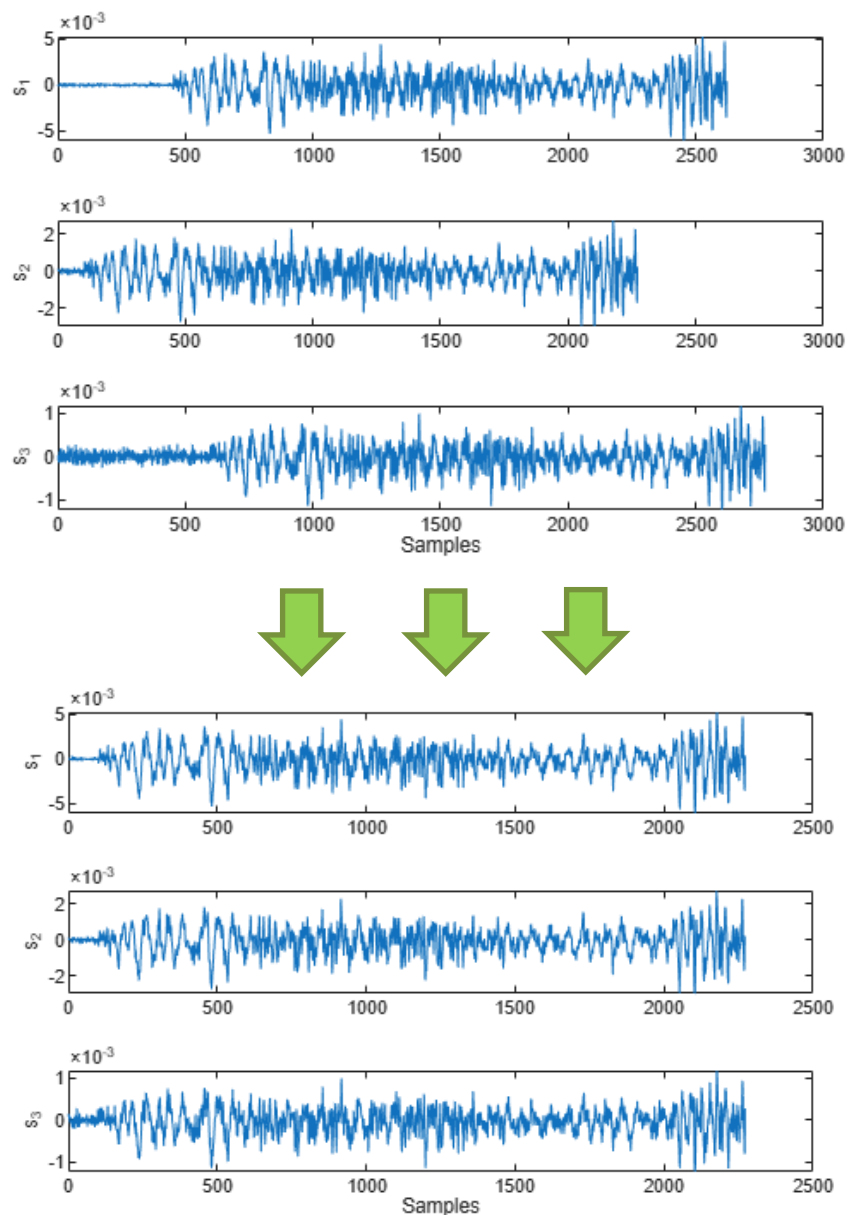
$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (\text{Eq. 5})$$

where:

- n is sample size
- x_i, y_i are the individual sample points indexed with i
- $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ (the sample mean); and analogously for $\bar{y}$

[Source - <https://forum.step.esa.int/t/offset-tracking-how-does-the-algorithm-operates/15365/10>]

A cross-correlation coefficient of 1.0 means perfect alignment and temporal integrity, proving no timing shifts occurred during the export process.



[Source - <https://www.mathworks.com/help/signal/ug/align-signals-using-cross-correlation.html>]

Data Integrity

Software-Specific Extraction Integrity (Python Integration) :-)

Since we are planed to do model training part using Python, we have to find how simulation tools interact with Python in different ways. Proving data extraction is reliable means proving that the hand-off between the simulation engine and the Python deep learning environment is completely flawless.

PSCAD / EMTDC (COMTRADE to Python)

PSCAD handles massive, microsecond-level transient data perfectly, but it relies on external files to move data.

- **Pros:** Perfectly captures the massive, microsecond-level high-frequency transient data needed for fault classification.
- **Cons:** Relies on external binary files (.DAT). When Python reads these files, incorrect byte-parsing can cause floating-point errors, which corrupts the voltage magnitudes.
- **Validation Required** a Cross-Correlation Analysis must be run between the raw PSCAD output graphs and the Python array (using libraries like numpy or a comtrade module) to prove 100% waveform preservation.

MATLAB / Simulink (Engine API Handoff)

MATLAB processes its data directly in its own RAM workspace.

- **Pros:** Using the MATLAB Engine API for Python allows zero-copy data transfer. This guarantees a 0% Packet Loss Rate (PLR) because the data moves directly from MATLAB's RAM into Python's RAM without ever touching the hard drive.
- **Cons:** If the API is not used, exporting large matrices to a disk first and reading them back into Python can introduce severe memory bottlenecks and packet drops.

OpenDSS (COM Interface Polling)

OpenDSS is driven entirely by Python through a Component Object Model (COM) interface.

- **Pros:** Allows direct control and data access entirely through Python scripting.
- **Cons:** Because Python has to "ask" OpenDSS for the data at every time step, minor CPU delays can cause Python to poll at irregular intervals. This leads to artificial clock drift and jitter.
- **Validation Required** strict time-step watermarking must be implemented in the Python script to ensure every single phase measurement aligns precisely with the exact millisecond of the simulation clock.

pandapower (Native Pandas Reliability)

pandapower is inherently built on top of Python, making the data hand-off seamless.

- **Pros:** Integrity risk is near zero. Because the simulation results are generated natively as Pandas DataFrames, there is no extraction or parsing required. Data completeness is guaranteed by design.
- **Cons:** This tool strictly lacks the high-frequency transient fidelity required for advanced relay fault classification.

Data Integration Summary

As citations I had followed, PSCAD is the best suited software with data integration with Python and it's a standard format uses in the world. Also, MATLAB/SIMULINK is the next best suited with Zero-Copy Transfer



PSCAD

In power system modeling, the PSCAD/EMTDC to Python pipeline provides the absolute highest physical realism for extracting deep learning fault signatures.

PSCAD captures the exact microsecond-level waveform distortions that occur during a physical short circuit. Validating this extraction through the standard COMTRADE binary format ensures a 100% reliable, real-world training dataset.

Proof of PSCAD's Superior Realism

Hong, J. et al. (2022). Deep-Learning based fault events analysis in power systems. Energies, 15(15), 5539.

[Source - <https://www.mdpi.com/1996-1073/15/15/5539/>]

- This paper explicitly details how PSCAD/EMTDC is used to generate highly realistic transient fault datasets and exports them via COMTRADE to train Convolutional Neural Networks (CNNs).

Proof for COMTRADE as the Real-World Standard

IEEE Power and Energy Society. IEEE Standard Common Format for Transient Data Exchange (COMTRADE) for Power Systems.

[source 1 - <https://standards.ieee.org/ieee/C37.111/3795/>]

[Source 2 - https://www.ni.com/docs/en-US/bundle/labview-electrical-power-toolkit/page/comtrade-file-format.html?srsltid=AfmBOopsfX87iKldhMPeSwuo8d_PMt2KUNYfKEZEdDEiKc9mp5gYL5Nf]

- IEEE standard proves that building a machine learning dataset on COMTRADE files guarantees 100% compatibility with real-world hardware relays.



MATLAB and Simulink

MATLAB/Simulink serves as the definitive runner-up for time-series extraction in fault classification architectures.

The primary architectural advantage is the MATLAB Engine API for Python facilitates zero-copy data transfer directly into deep learning memory arrays. However, because MATLAB is a generalized mathematical modeling environment rather than a dedicated EMT simulator like PSCAD, its high-frequency transient component libraries are not as specialized.

Proof for MATLAB can Generate Transients

Guo, M. F. et al. (2017). Deep-learning-based earth fault detection using continuous wavelet transform and convolutional neural network. IEEE Sensors Journal, 18(3), 1291-1300.

[Source - <https://ieeexplore.ieee.org/document/8116617>]

- This highly cited paper proves that MATLAB/Simulink is an industry-accepted tool for generating distribution network fault transients specifically intended to train CNNs.

Proof for the MATLAB Engine API (Zero-Copy Transfer)

MathWorks. (2023). MATLAB Engine API for Python.

[Source - <https://www.mathworks.com/help/matlab/matlab-engine-for-python.html>]

- Citing official software documentation validates the claim that MATLAB can pass matrices directly into Python's active memory (RAM), eliminate the need for slow text-file exports like CSV.